

CASO PRÁCTICO 1 - José Cruces Aparicio

UD 6 - 2º DAM- Diagrama de clases UML

1. Introducción

La empresa necesita una aplicación sencilla de control de acceso de visitantes que funcione en una tablet situada en la entrada del recinto. El objetivo es registrar quién entra, de qué empresa procede y a qué empleado viene a visitar, de forma que el personal de seguridad pueda consultar, validar y cerrar las visitas con facilidad.

Para modelar esta solución se propone un diagrama UML de clases con tres clases principales:

Visitante

Empleado

RegistroVisita

Esta estructura permite representar tanto los datos persistentes como las relaciones entre las entidades principales del sistema. En UML, las asociaciones entre clases y sus multiplicidades permiten expresar cuántas instancias de una clase pueden relacionarse con otra. Ese enfoque está recogido en la especificación de UML de OMG y en documentación técnica de IBM sobre diagramas de clases y multiplicidad.

2. Análisis de necesidades

La aplicación debe permitir:

- Registrar un visitante.
- Identificar a qué empleado visita.
- Guardar la fecha y hora de entrada.
- Confirmar el acceso.
- Registrar la salida del visitante.
- Consultar visitas activas o históricas.

A partir de estas necesidades, la información persistente más importante es:

- Datos del visitante.
- Datos del empleado visitado.
- Datos concretos de cada visita.

3. Definición de clases, propiedades y métodos

Clase Visitante

Representa a la persona externa que accede al recinto.

Propiedades

1. **idVisitante: int**
Identificador único del visitante.
2. **nombreCompleto: String**
Nombre y apellidos del visitante.
3. **empresa: String**
Empresa u organización de procedencia.
4. **documentoIdentidad: String**
DNI, NIE, pasaporte u otro documento identificativo.
5. **telefonoContacto: String**
Número de teléfono del visitante.
6. **correoElectronico: String**
Correo de contacto, útil si se requiere seguimiento.

Métodos

1. **registrarDatos()**
Permite guardar los datos del visitante en el sistema.
2. **actualizarDatos()**
Permite modificar información del visitante si hay errores o cambios.
3. **mostrarInformacion()**
Devuelve o muestra los datos principales del visitante.

Funcionalidad

La clase Visitante almacena la información personal y corporativa básica necesaria para identificar a la persona externa que solicita acceso.

Clase Empleado

Representa a la persona de la empresa que recibirá la visita.

Propiedades

1. **idEmpleado: int**
Identificador único del empleado.
2. **nombreCompleto: String**
Nombre y apellidos del empleado.
3. **departamento: String**
Área o departamento al que pertenece.
4. **cargo: String**
Puesto de trabajo del empleado.

5. **extensionTelefono: String**
Extensión telefónica interna.
6. **correoCorporativo: String**
Correo electrónico de la empresa.

Métodos

1. **autorizarVisita()**
Permite confirmar que la visita está permitida.
2. **rechazarVisita()**
Permite indicar que la visita no debe acceder.
3. **consultarVisitasProgramadas()**
Permite revisar visitas asociadas al empleado.

Funcionalidad

La clase Empleado identifica a la persona interna que recibe la visita y participa en la validación o seguimiento del acceso.

Clase RegistroVisita

Representa cada evento de visita. Es la clase central del sistema porque relaciona un visitante con un empleado en una fecha y estado concretos.

Propiedades

1. **idRegistro: int**
Identificador único de la visita.
2. **fechaVisita: Date**
Fecha del registro de la visita.
3. **horaEntrada: Time**
Hora de entrada del visitante.
4. **horaSalida: Time**
Hora de salida del visitante.
5. **estadoVisita: String**
Estado de la visita: pendiente, autorizada, en curso, finalizada o cancelada.
6. **motivoVisita: String**
Breve descripción del motivo de la visita.
7. **observaciones: String**
Campo libre para incidencias o anotaciones del personal de seguridad.

Métodos

1. **crearRegistro()**
Genera una nueva visita y la asocia con un visitante y un empleado.
2. **confirmarEntrada()**
Cambia el estado de la visita cuando el visitante accede al recinto.
3. **registrarSalida()**
Guarda la hora de salida y marca la visita como finalizada.
4. **cancelarVisita()**
Permite anular el registro si la visita no se realiza.

Funcionalidad

La clase RegistroVisita gestiona el ciclo de vida completo de una visita y enlaza a las otras dos clases.

4. Relaciones y asociaciones entre clases

La estructura de relaciones propuesta es la siguiente:

Relación entre Visitante y RegistroVisita

- Un Visitante puede tener 0..* registros de visita.
- Cada RegistroVisita pertenece a 1 único visitante.

Tipo de relación: asociación.

Justificación: una misma persona puede visitar la empresa varias veces en fechas distintas.

Relación entre Empleado y RegistroVisita

- Un Empleado puede recibir 0..* registros de visita.
- Cada RegistroVisita está asociada a 1 empleado.

Tipo de relación: asociación.

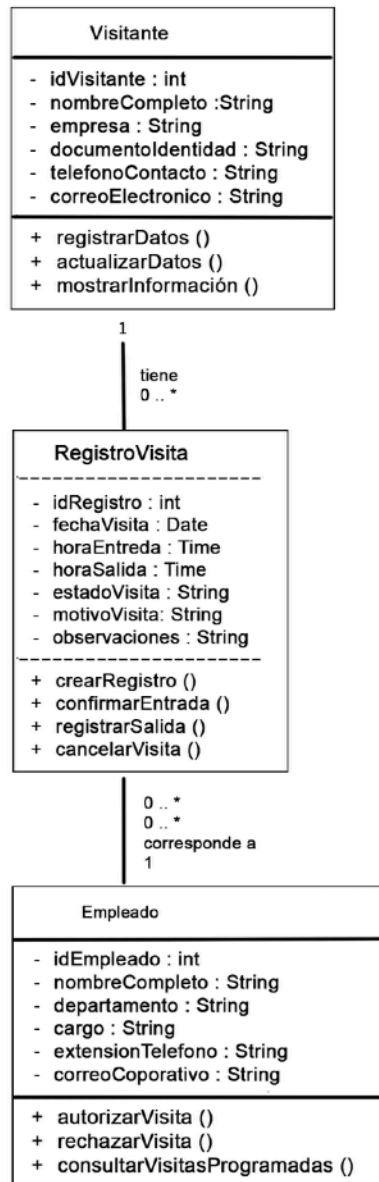
Justificación: un empleado puede recibir muchas visitas, pero cada registro concreto corresponde a un único empleado visitado.

Tipo de dependencia lógica

La clase **RegistroVisita** actúa como clase intermedia del modelo. No tendría sentido crearla sin asociarla a un visitante y a un empleado. Por ello, aunque en UML lo representamos mediante asociaciones normales, desde el punto de vista funcional es la entidad dependiente del proceso.

Las multiplicidades en UML sirven precisamente para expresar estas cantidades de participación entre clases, por ejemplo 1, 0..1, 0..* o 1..*.

5. Diagrama UML de clases



Interpretación del diagrama

- Visitante 1 — 0..* RegistroVisita
- Empleado 1 — 0..* RegistroVisita

Es decir un visitante puede aparecer en muchos registros, un empleado puede recibir muchas visitas, y cada registro pertenece a un único visitante y a un único empleado.

6. Justificación del diseño

Se ha optado por una tercera clase llamada **RegistroVisita** porque evita mezclar información temporal de la visita con los datos permanentes del visitante o del empleado.

Este diseño es adecuado porque:

- separa claramente las responsabilidades;
- facilita la persistencia de datos;
- permite registrar múltiples visitas históricas;
- hace más fácil la consulta de visitas activas y finalizadas;
- mejora la escalabilidad de la aplicación.

Desde un punto de vista profesional, este modelo es más correcto que guardar toda la información en una sola clase, porque respeta mejor la organización lógica de los datos y simplifica el mantenimiento futuro del sistema.

7. Manual de usuario

7.1 Finalidad del manual

Este manual está dirigido al personal de seguridad o recepción encargado de utilizar la tablet de control de accesos.

Su objetivo es explicar, de forma sencilla, cómo usar la aplicación para registrar la entrada y salida de visitantes.

Las guías de redacción de procedimientos recomiendan instrucciones claras, ordenadas y orientadas a la acción, con pasos secuenciales fáciles de seguir.

7.2 Requisitos previos

Para utilizar la aplicación se necesita:

- una tablet encendida;
- acceso a la aplicación de control de visitas;
- conexión a la red interna o a la base de datos, si procede;
- datos básicos del visitante y del empleado visitado.

7.3 Pantalla principal

La pantalla principal mostrará las opciones:

- **Nueva visita**
- **Consultar visitas**
- **Registrar salida**
- **Buscar empleado**
- **Cerrar sesión**

7.4 Procedimiento para registrar una nueva visita

Paso 1. Abrir la opción “Nueva visita”

El personal de seguridad pulsa el botón **Nueva visita**.

Paso 2. Introducir los datos del visitante

Se rellenan los siguientes campos:

- nombre completo;
- empresa;
- documento de identidad;
- teléfono;
- correo electrónico;
- motivo de la visita.

Paso 3. Seleccionar al empleado visitado

Se busca al empleado por:

- nombre;
- departamento;
- cargo.

Una vez localizado, se selecciona en la lista.

Paso 4. Confirmar el registro

El sistema crea el registro de visita con:

- fecha actual;
- hora de entrada;
- estado inicial.

Paso 5. Verificar acceso

Si la visita es válida, el sistema o el empleado la autoriza y la visita pasa al estado **autorizada** o **en curso**.

7.5 Procedimiento para consultar visitas

1. Pulsar **Consultar visitas**.
2. Filtrar por nombre del visitante, empleado, fecha o estado.
3. Revisar la información en pantalla.
4. Seleccionar una visita concreta para ver su detalle.

7.6 Procedimiento para registrar la salida

1. Pulsar **Registrar salida**.
2. Buscar al visitante activo.
3. Seleccionar el registro correspondiente.
4. Confirmar la salida.
5. El sistema guarda la hora de salida y cambia el estado a **finalizada**.

7.7 Gestión de incidencias frecuentes

Error en los datos del visitante

- Buscar al visitante.
- Editar la información errónea.
- Guardar los cambios.

El empleado no aparece en la lista

- Revisar ortografía del nombre.
- Buscar por departamento.
- Verificar si el empleado está dado de alta en el sistema.

La visita se cancela

- Abrir el registro.
- Seleccionar la opción **Cancelar visita**.
- Añadir observación si es necesario.

7.8 Recomendaciones de uso

- Comprobar siempre la identidad del visitante antes de registrar el acceso.
- Verificar que el empleado seleccionado es el correcto.
- Registrar la salida en cuanto el visitante abandone el recinto.
- Anotar incidencias relevantes en observaciones.

8. Guía de referencia

8.1 Objetivo de la guía

La guía de referencia describe de forma resumida el funcionamiento de la aplicación y sus elementos principales. No incluye código fuente ni especificaciones técnicas profundas, pero sí una demostración funcional del sistema.

8.2 Descripción general del sistema

La aplicación consta de tres bloques lógicos:

Gestión de visitantes

Permite almacenar y consultar los datos personales y de empresa de cada visitante.

Gestión de empleados

Permite identificar a la persona interna que recibe la visita.

Gestión de registros de visita

Permite crear, actualizar, cerrar y consultar cada acceso.

8.3 Funciones principales

- Alta de visitante.
- Asociación del visitante con un empleado.
- Registro de entrada.
- Confirmación o autorización.
- Registro de salida.
- Consulta histórica de visitas.

8.4 Estructura de información persistente

La información persistente queda organizada en tres entidades:

- **Visitante**: datos personales.
- **Empleado**: datos corporativos.
- **RegistroVisita**: datos del evento de acceso.

Este modelo es adecuado para base de datos relacional, ya que RegistroVisita puede almacenar claves foráneas hacia Visitante y Empleado.

8.5 Demostración práctica

Ejemplo de uso real

Supongamos el siguiente caso:

- **Visitante:** Laura Gómez Pérez
- **Empresa:** Soluciones Técnicas S.L.
- **Documento:** 12345678X
- **Empleado visitado:** Carlos Martín Ruiz
- **Departamento:** Sistemas
- **Motivo:** Reunión de mantenimiento

Demostración paso a paso

1. Alta del visitante

El personal de seguridad introduce los datos de Laura Gómez Pérez.

2. Selección del empleado

Se busca a Carlos Martín Ruiz y se selecciona como empleado visitado.

3. Creación del registro

El sistema genera:

- fecha: 24/03/2026
- hora de entrada: 09:15
- estado: autorizada

4. Desarrollo de la visita

La visita permanece activa durante la reunión.

5. Registro de salida

A las 10:40 se registra la salida.

Resultado final del registro

- visitante: Laura Gómez Pérez
- empleado: Carlos Martín Ruiz
- entrada: 09:15
- salida: 10:40
- estado: finalizada

8.6 Ventajas del sistema propuesto

El sistema propuesto destaca, en primer lugar, por ofrecer un control de accesos básico pero eficaz, suficiente para cubrir las necesidades esenciales de seguridad sin introducir una complejidad innecesaria. Esta simplicidad no implica una pérdida de funcionalidad, sino más bien una solución equilibrada que permite gestionar quién entra y sale de manera ordenada y fiable.

A su vez, proporciona una adecuada trazabilidad de las entradas y salidas, lo que resulta fundamental para mantener un registro histórico de los movimientos. Esta capacidad no solo facilita la supervisión diaria, sino que también permite consultar información en caso de incidencias o auditorías, aportando transparencia y control.

Otro de sus puntos fuertes es la facilidad de uso, especialmente pensada para personal no técnico. La interfaz y el funcionamiento están diseñados para ser intuitivos, reduciendo la necesidad de formación específica y favoreciendo una rápida adopción por parte de los usuarios.

Por último, el sistema cuenta con una estructura clara y bien organizada que sienta las bases para futuras ampliaciones. Esto significa que, a medida que surjan nuevas necesidades, será posible incorporar funcionalidades adicionales sin tener que rediseñar completamente la solución, garantizando así su evolución y sostenibilidad a largo plazo.

8.7 Posibles mejoras futuras

Desde una perspectiva crítica y propositiva, el sistema no debería limitarse únicamente a registrar accesos, sino aspirar a convertirse en una solución más completa, inteligente y adaptada a las necesidades reales de seguridad y gestión. En este sentido, su evolución podría orientarse hacia la incorporación de tecnologías que aporten mayor fiabilidad, automatización y valor añadido.

Por ejemplo, la integración de un lector de DNI o de códigos QR permitiría agilizar la identificación de los visitantes, reduciendo errores manuales y mejorando la experiencia de acceso. A esto se podría sumar la implementación de una firma digital, que no solo reforzaría la validez del registro, sino que también aportaría un componente legal y de trazabilidad más sólido.

Asimismo, la inclusión de una fotografía en el momento del acceso añadiría una capa adicional de seguridad, facilitando la verificación visual y el control posterior en caso de incidencias. Este proceso podría complementarse con un sistema de aviso automático al empleado visitado, optimizando la comunicación interna y evitando esperas innecesarias en recepción.

Desde el punto de vista de la gestión, la generación automática de informes diarios o mensuales permitiría analizar patrones de visitas, detectar posibles anomalías y mejorar la toma de decisiones. Finalmente, la integración con tornos o puertas automáticas supondría un paso hacia la automatización completa del acceso, reforzando tanto la eficiencia como el control del flujo de personas.

En conjunto, estas mejoras no solo amplían las funcionalidades del sistema, sino que reflejan una visión más innovadora y creativa, alineada con los criterios de calidad y valor añadido que se valoran en la rúbrica.

9. Conclusión

El modelo propuesto resuelve de forma clara las necesidades de una aplicación de control de acceso de visitantes. Las tres clases definidas —Visitante, Empleado y RegistroVisita— separan correctamente la información persistente y permiten representar las relaciones esenciales del sistema.

Además, el manual de usuario facilita el uso por parte del personal de seguridad, mientras que la guía de referencia resume el funcionamiento de la aplicación y muestra una demostración práctica coherente.

En conjunto, la propuesta cumple los objetivos del caso: documentar la estructura de la información persistente y elaborar documentación útil para el uso de la aplicación. La notación UML empleada sigue el enfoque estándar de clases, asociaciones y multiplicidades descrito por la especificación UML y por documentación técnica de apoyo.

10. Bibliografía y recursos consultados

- Object Management Group (OMG). **Unified Modeling Language (UML)**, especificación oficial.
- IBM Developer. **The UML 2 class diagram**. Explica asociaciones, roles y multiplicidades en diagramas de clases.
- IBM Documentation. **Setting the multiplicity property in UML diagrams**. Referencia sobre multiplicidad entre clases.
- Microsoft Learn. **Writing step-by-step instructions**. Recomendaciones para redactar procedimientos claros.
- Microsoft Learn. **Microsoft Writing Style Guide**. Buenas prácticas para documentación técnica y manuales.
- Unidad 6 Interfaces